

PRODUCT SPECIFICATION

MeshCDN

A self-hosted, leaderless distributed CDN in which the configuration *is* the command log — operated entirely from a chat window.

Release v4.3.0

Runtime Single static Go binary (`cdn-agent`) + OpenResty, per node

Platform Linux / systemd / amd64 · Debian, Ubuntu, RHEL family

Control plane Telegram bot · local CLI · mesh broadcast — one command language across all three

Document date August 2026

14.1k

LINES OF GO

58

SOURCE FILES

3

DIRECT DEPS

0

CONTROL SERVERS

~5 MB

DEPLOYED BINARY

1 What MeshCDN Is

MeshCDN turns a handful of ordinary VPS instances into a coherent content-delivery network with no control-plane server, no web console, and no orchestration layer. Every node holds the complete configuration, every node can accept and broadcast changes, and the whole system is driven by a three-verb command language that doubles as its own backup format.

A production deployment consists of N identical nodes. Each runs OpenResty for the data path and a single Go binary, `cdn-agent`, for the control path. The agent parses commands, owns a local SQLite configuration database, generates the entire nginx configuration, manages TLS certificates end to end, gossips with its peers over the same port that serves customer traffic, and — on exactly one node at a time — speaks to the Telegram API so an operator can run the cluster from a phone.

The design target is the underserved middle: 10–50 nodes and 50–500 domains, operated by one person or a small team that wants CDN behaviour without a commercial CDN's pricing, opacity, or lock-in — and specifically including operators whose nodes sit inside networks where reaching a foreign control plane is unreliable.

1.1 The problems it was built to solve

Problem	MeshCDN's answer
Control planes are single points of failure	There is no control plane. Nodes are peers; the "bot node" is merely whoever is currently answering the phone, and that role is a single overridable field.
Config drift across nodes	A monotonic cluster-wide <code>config_version</code> . Nodes that fall behind pull a full snapshot and replay it — convergence by construction rather than by careful delta bookkeeping.
Backups that don't restore	The backup format is the command language. Restoring a node means replaying a text file that a human can read, diff, and hand-edit.
Management interfaces are attack surface	No web panel, no public API, no management port. Mesh traffic is multiplexed onto the CDN's own HTTPS port and is externally indistinguishable from ordinary requests.
Nodes behind restricted networks	Only the bot node ever contacts Telegram. Every other node reaches only its peers, over standard HTTPS.
Certificate management is fiddly	ACME issuance, multi-SAN renewal, upload, and self-signed fallback unify behind one selection algorithm and one renewal loop. IP certificates are not a special case.
"It worked when I deployed it"	Cluster upgrades are verified against peer heartbeats and roll back automatically on an unhealthy restart, instead of reporting success at dispatch time.

2 Design Philosophy

These ten principles are not retrospective narrative. They are written into the project's design document as its "constitution," and every implementation decision is required to trace back to one of them. Several features were deliberately deleted between major versions for violating one.

1 Configuration is command; command is backup; backup is sync.

There is no second representation of state. The database is a materialised view of a command sequence; export dumps that sequence back out as commands; restore replays it; inter-node synchronisation ships it. One artifact serves four roles, so the four can never disagree.

2 The command language is the product; the terminal is a shell.

Telegram is the default front-end, not the system. The same command text executes identically through the chat bot, through `cdn-agent exec` over SSH, and through a mesh broadcast from a peer. Front-ends are replaceable; the instruction set is the contract.

3 Three verbs, four segments, no exceptions.

Every operation is `write`, `delete`, or `view`, expressed as exactly four whitespace-separated segments. Empty positions are filled with `-` rather than omitted. A parser with no special cases is a parser with no ambiguity, and — crucially — `w` and `d` are textual mirrors, so undoing a change means retyping it with one character altered.

4 Nodes are equals. There is no master.

No election, no failover, no split-brain protection — because there is no role to contend for. Every node holds the full configuration and can execute and broadcast. Removing leader election removes an entire category of distributed-systems failure, at the cost of accepting eventual rather than strong consistency.

5 Precision is priority.

Overlapping rules are never resolved by hand-assigned weights. Ordering falls out of specificity: `exact` > `prefix` > `regex` > `wildcard`; longer CIDR prefix wins. Operators describe intent; the system derives the order. This also means a rule's behaviour does not change when an unrelated rule is added.

6 A batch is the atomic unit; failures never enter the protocol.

A multi-line submission executes in order, failures are skipped and reported, and only the successful subset advances `config_version` — inside a single SQLite transaction with the write itself. Peers therefore can only ever pull a slice of history that is known to have succeeded. There is no "partially applied" state to reconcile.

7 The program's boundary is the information boundary.

The software's duty is to tell the operator the truth; the decision is the operator's. Alerting is mandatory; papering over failures is not. This principle was adopted explicitly to `delete` code: several layers of silent fallback were removed in v4 because a system that quietly compensates is a system whose real state you cannot know. The only surviving exceptions are the minimum needed to keep a node bootable and reachable.

8 Bot → node is one-way; node ↔ node is two-way.

The bot's only job is to receive. Nodes that are not the bot node never open a connection to Telegram, which keeps them functional in networks where that connection is unavailable. Peer traffic is plain HTTPS on the CDN port.

9 The group chat is the console — and the audit log.

Commands are accepted only in a designated group; direct messages are refused. Operational history is therefore recorded, timestamped, and attributed by the chat platform, at zero implementation cost.

10 Minimum attack surface.

No web UI, no SSH dependency, no public API, no management port. Cluster membership is proved with `sha256(group_id + bot_token)`, so a new node can join by presenting a secret to *any* live peer — the bot does not need to be online for the cluster to grow.

DELETION AS A DESIGN ACT

Version 4 removed rule templates, domain groups, three parallel version streams, the "never switch once selected" certificate rule, the special-casing of IP certificates versus domain certificates, and several defensive fallback layers. Each was removed because it conflicted with a principle above. The v4 design document maintains an explicit table of these deletions — the system is defined as much by what it refuses to carry as by what it implements.

3 Architecture: Skeleton, Muscle, Blood

The codebase is organised by lifecycle rather than by layer. Every file belongs to one of three systems, distinguished by how often it changes and what happens to it during an upgrade. Package boundaries are physical boundaries: exactly one package owns each on-disk artifact.

3.1 Skeleton — what sits still on disk

Node state is split across two directories with opposite guarantees. This split is the entire upgrade strategy:

```
/etc/meshcdn/  
├── persistent/      ← survives every upgrade; exactly four items  
│   ├── identity.json  node identity, bot token, group id, AI provider config  
│   ├── peers.json    cluster member IPs and join order  
│   ├── certs/        all certificates + manifest.json  
│   └── snapshot.cmd   the live configuration, as replayable command text  
└── runtime/         ← deleted wholesale on upgrade, rebuilt from persistent/  
    ├── config.db      SQLite configuration database  
    ├── logs.db        per-minute traffic aggregates  
    ├── nginx/         generated OpenResty configuration  
    ├── cache/ logs/ challenges/ welcome/ tmp/
```

An upgrade is therefore mechanical and total: delete `runtime/`, stop the service, replace the binary, start the service, and let the agent rebuild everything derived from the four persistent files. Nothing accumulates across versions, so no migration code is needed and no version can inherit another's corrupted derived state. The corollary is a genuine forward-compatibility guarantee: any future version that can read those four file formats can adopt a v4 node.

3.2 Muscle — the command system

Every input path — chat message, CLI invocation, mesh broadcast — converges on `internal/command`. The package defines the parsed `Command`, the `Handler` interface each resource type implements, the `Effects` a command declares, and the batch transaction executor. Handlers are registered in a registry keyed by type name.

The architectural promise is that adding a feature means adding one handler file and one registry line. Verb dispatch, transaction handling, version bumping, snapshot export, peer notification, and error classification are written once and never touched again. Handlers also never touch the filesystem or the network directly — they return declarative `Effects` (reload nginx, purge cache, broadcast, force a version bump) that the executor applies after the transaction commits. That separation is what makes a handler testable and what keeps side effects from escaping a rolled-back transaction.

3.3 Blood — the runtime loops

Four independent loops run inside the agent. They never call one another; they communicate only through channels and the database, so a stalled or crashed loop cannot take another down.

Loop	Cadence	Responsibility
Heartbeat	60 s	Ping every peer; exchange <code>config_version</code> , <code>program_version</code> , and known-peer count; record measured RTT locally. A version lag triggers a pull.
Certificate scan	6 h	Inspect every certificate; anything with under 3 days remaining enters the renewal pipeline.
Event stream	continuous	Handle non-config messages: expiry alerts, ACME challenge sharing, certificate distribution, cluster notices.
Stats collector	60 s	Tail nginx's JSON stats log incrementally, aggregate per minute into <code>logs.db</code> , persist the read offset, expire data older than 7 days.

4 The Command Language

4.1 Strict four-segment form

```

/<verb> <type> <scope> <params>

/w domain    https://a.com:443    https://10.0.0.4:443
/w cache     img-7d              patterns=*.jpg,*.png ttl=604800
/w bind      a.com               cache=img-7d
/d cache     img-7d              patterns=*.jpg,*.png ttl=604800
/v stats     a.com               24h

```

The parser expects four segments always. `-` is the universal placeholder meaning "default / none / unscoped," and in a domain field it carries a second meaning that makes fleet-wide commands possible: *this machine's own public IP*. One identical command line, broadcast to every node, resolves to a different address on each — and `-` is stored verbatim in the database, substituted only at nginx-generation time, so the snapshot stays portable across nodes.

4.2 Mirror symmetry

`w` and `d` share the parse and validate paths and diverge only at execution. Each type declares which fields constitute its primary key; a repeated write to the same key is an overwrite, which collapses "add" and "modify" into a single operation. Deletion matches on the primary key alone, so non-key parameters may be present or absent in a `d` command — meaning an operator can delete a rule by copying its line out of the export and changing `w` to `d`.

4.3 Three command classes plus cluster verbs

Class	Shape	Purpose
A — direct	/w domain · ssl · sslfile	Scope is a real business object: a host:port endpoint, a domain, an IP.
B — objects	/w cache · defense · redirect · header	Scope is an object <i>name</i> . Defines a reusable, unbound policy.
C — bindings	/w bind <scope> <type>: <name>	Attaches a policy object to a domain or IP endpoint. Many-to-many.
Cluster	/v export · status · stats · nodes · cache	Read-shaped inspection and operational actions.
System	/sync · /target · /upgrade · /confirm · /menu · /help	Standalone verbs for cluster-level actions.

The object/binding split replaced an earlier template mechanism. It buys reuse without inheritance: one cache policy can serve two hundred domains, one domain can carry a dozen policies, and changing the policy changes every consumer at once. It also makes deletion safe — removing a bound object is refused, or requires explicit confirmation for a cascading delete.

4.4 Precision-ordered matching, delegated to nginx

When several objects bind to one domain, conflicts resolve by specificity, not by binding order:

```
/w cache broad      patterns=*.jpg,*.png    ttl=604800
/w cache narrow     patterns=hero.jpg      ttl=86400
/w bind a.com       cache:broad
/w bind a.com       cache:narrow

GET /hero.jpg → narrow wins (exact) → cached 1 day
GET /other.jpg → broad wins (wildcard) → cached 7 days
```

The agent does not implement a runtime matching engine. It sorts patterns by precision and emits them as ordered nginx `location` blocks — `location =` for exact, `^~` for prefix, `~*` for regex, a wildcard catch-all last — and lets nginx's own matching engine do the work. Writing a matcher would have meant owning a second, subtly different implementation of behaviour the data path already provides.

4.5 Merging equal-precision conflicts: strictness wins

When two objects specify the same precision for the same pattern, the values merge automatically toward the stricter setting rather than erroring, prompting, or taking the first match:

Field type	Rule
TTL	Minimum wins — shorter caching is safer.
Body-size limit	Minimum wins, after normalising <code>k / m / g</code> suffixes to bytes.
Boolean hardening (HSTS, ...)	Logical OR — enabled is stricter.
Action (block / allow)	<code>block</code> wins.

A BUG THIS RULE CAUGHT

Size limits were originally merged first-wins by object name, which meant an object named `a-lenient` with `size=1g` could silently override `b-strict` with `size=10m` — alphabetical order deciding a security control. The fix normalises units and takes the minimum, restoring the invariant that merging can only ever tighten. The class of bug is the point: any merge rule that depends on input order is a merge rule that will eventually surprise its operator.

4.6 Closed key sets and confirmation gates

Every B-class object validates its parameter keys against a closed allow-list; an unrecognised key is a parse-time error. This replaced silent dropping, where a typo'd key produced a cheerful success message and no effect. Two parameters that are designed but unimplemented — `geo=` and `cc=` — return an explicit "not implemented" rather than pretending to work. Principle 7, enforced at the parser.

Genuinely dangerous operations are held rather than refused. A port-protocol conflict or a cascading delete produces a `PendingConfirmation` with a short ID, an in-memory five-minute TTL, and a message explaining what will happen; replying `/confirm <ID>` re-executes the original command in confirmed mode. The state is deliberately not persisted — a restart or a timeout simply means retyping the command, which is the correct failure mode for a destructive action.

5 Feature Catalogue

Status legend: **SHIPPED** in v4.3.0 **PARTIAL** implemented within a stated scope **PLANNED** designed, not built.

5.1 Traffic and routing

Capability	Status	Detail
Reverse proxy	SHIPPED	<code>/w domain <host:port> <origin></code> . Origins are full URLs including scheme and port; upstream TLS uses SNI with TLS 1.2/1.3. A domain may be registered with no origin at all, for pure-redirect endpoints.
Multi-port, multi-domain	SHIPPED	Comma syntax expresses many-to-one in a single line: <code>a.com,b.com</code> and <code>80,443</code> . Newlines express independent entries; the whole block is one atomic batch.
Cluster-unique port protocol	SHIPPED	A dedicated table binds each port to exactly one protocol cluster-wide, updated implicitly by domain writes. Contradicting it raises <code>PORT_CONFLICT</code> and requires confirmation — preventing a node-local override from silently diverging one node's listener map from the rest.
IP endpoints	SHIPPED	An IP is a domain. Registered IPs get full endpoint configuration and certificate selection; unregistered ones fall to a default server with a self-signed certificate and a 200-status branded page — never a connection error.
Caching	SHIPPED	<code>patterns</code> , <code>ttl</code> , <code>browser_ttl</code> , <code>hsts</code> . Shared 50 MB key zone with two-level disk storage.
Cache purge	SHIPPED	<code>/v cache - purge-all</code> clears locally, reloads, then fans <code>purge-node</code> to every peer; peers do not re-broadcast, so there is no storm. Full-clear only — OpenResty keys by content hash, so pattern-scoped purging cannot be done honestly. Deliberately read-shaped: purging is not configuration and must not consume a version number.
Redirects	SHIPPED	<code>from</code> , <code>to</code> , <code>status</code> , <code>regex</code> . Precision-ordered alongside cache rules.
Header rewriting	SHIPPED	Add and delete, on both request and response.
Smart origin routing	PLANNED	Node-to-node relay back to origin. Blocked on design sign-off for per-hop fallback: without automatic direct-to-origin failover, relaying introduces a new single point of failure. Explicitly not started rather than half-built.

5.2 Protection

Capability	Status	Detail
IP allow / block	SHIPPED	CIDRs and bare addresses, ordered by prefix length. Emitted as native nginx <code>allow / deny</code> .
User-Agent filtering	SHIPPED	All UA regexes bound to one server merge into a <i>single</i> <code>access_by_lua_block</code> — one loop per request, not one block per rule. Runs after <code>allow / deny</code> , so block-beats-allow holds naturally: an IP-denied request never reaches the UA check.
Regex safety gate	SHIPPED	Patterns are validated at parse time and rejected for control characters, invalid syntax, or catastrophic-backtracking shapes such as <code>(a+)+</code> . Validation runs on Go's RE2 engine while execution is PCRE — an intentional asymmetry, since RE2's refusal of backreferences and lookahead also excludes the constructs that make ReDoS possible. A bad pattern can never reach generated configuration.
Rate limiting	SHIPPED	Per-source-address zone with configurable burst.
Referer rules, body-size caps	SHIPPED	Per-object; size caps merge toward the smaller value.
GeoIP blocking / CC sliding window	PLANNED	Keys reserved, parse with an explicit error. GeoIP needs a database-selection and update policy the operator must agree to own; CC protection needs a shared-memory window design.

5.3 Certificate subsystem

Three sources — Let's Encrypt, operator upload, self-signed — are metadata labels on one uniform pipeline, and source *is* precision: LE > upload > self-signed.

Capability	Status	Detail
ACME HTTP-01 issuance	SHIPPED	Responsibility is deterministic without coordination: <code>candidates(D) = peers</code> whose IP appears in D's DNS; the issuer is <code>candidates[hash(D) % len]</code> . DNS changes reassign automatically. Before validating, the responsible node broadcasts the challenge token to every candidate, so validation succeeds whichever node the CA happens to reach.
IP certificates	SHIPPED	Built on Let's Encrypt's short-lived IP certificates. Handled by the same commands and code path as domains — an empty-CN CSR with the short-lived profile is the only divergence.
Upload via chat	SHIPPED	Dropping a <code>.crt</code> / <code>.key</code> into the group works without a command: the bot sniffs PEM content to decide which slot the file fills, extracts the CN to infer scope, and buffers the pair until both halves arrive.
Selection algorithm	SHIPPED	Among unexpired certificates covering an endpoint: rank by source, then by latest expiry. The selection is sticky while valid and re-evaluated immediately on expiry — stability without staleness.
Multi-SAN renewal	SHIPPED	Renewal identifiers are <code>{Subject} u SAN</code> , deduplicated. An earlier implementation renewed against the CN alone, quietly shrinking a multi-domain certificate to one name and dropping the rest to the next-priority certificate — often self-signed, producing browser warnings on domains nobody had touched.
Renewal pipeline	SHIPPED	Under 3 days remaining: attempt renewal (ACME or local regeneration) → on failure, look for a non-self-signed alternative with a later expiry → on failure, alert to Telegram with the raw ACME error, deduplicated for 24 h. There is no fourth step. Expiry is then handled by the selection algorithm as an ordinary state, not an emergency.
Distribution	SHIPPED	LE and uploaded certificates replicate cluster-wide over the config stream. Self-signed certificates never replicate — they are node-local fallbacks by definition.
Content-addressed storage	SHIPPED	Files are named by SHA-256 prefix with a <code>manifest.json</code> index carrying subject, SAN list, source, issuer, validity window, and current bindings. Identical certificates deduplicate; a renamed domain never orphans a file.
DNS-01	PLANNED	Deferred to a later release; requires provider API integration the operator must choose.

5.4 Cluster operations

Capability	Status	Detail
Join by shared secret	SHIPPED	A new node presents <code>sha256(group_id + bot_token)</code> to any live peer. That peer records the membership change as a versioned command, returns the current snapshot, and the joiner replays it and enters the heartbeat mesh. The bot need not be online.
Membership in the config stream	SHIPPED	Join and leave are ordinary commands that bump the version, so the peer list converges through the same machinery as everything else — no separate membership protocol to get wrong.
Push + heartbeat sync	SHIPPED	After a batch commits, the executing node pushes a version notification to all peers; the 60-second heartbeat independently detects lag. Both run always. Push is fast and lossy, heartbeat is slow and certain — typical convergence under 1 s, worst case under 2 minutes.
Snapshot pull with RTT selection	SHIPPED	A lagging node filters peers to those strictly ahead of it, picks the lowest measured RTT, pulls the full command list plus target version, then clears, replays, and bumps in one transaction. It never depends on the bot node, and a torn pull leaves no partial state.
Verified cluster upgrade	SHIPPED	See §6 — the most instructive fix in the project's history.
Bot role transfer	PARTIAL	<code>/v target <ip></code> validates the peer, writes a <code>bot_node_ip</code> override, and forces a version bump so a read-shaped action still propagates. The effective bot is "override, else lowest join order" — one predicate shared by status display, alert routing, and the boot gate, and it round-trips through the snapshot as <code>/w internal set-bot</code> . <i>Scope boundary</i> : the target begins polling Telegram after its next agent restart. Restart-free handover requires a supervisor refactor and was consciously deferred rather than faked.
Manual bot takeover	SHIPPED	If the bot node dies, SSH to any surviving node and point the override at it. No automatic drift, no split-brain arbitration — chosen deliberately, since a wrongly-triggered automatic handover is worse than a two-minute manual one.
Cross-node stats aggregation	PLANNED	Per-node statistics ship; cluster roll-up is designed and deferred.
DNS health auto-eviction	PLANNED	Requires committing to specific DNS provider APIs.

5.5 Observability

Capability	Status	Detail
Traffic statistics	PARTIAL	<code>/v stats</code> over an arbitrary window (<code>24h</code> , <code>7d</code> , ...) with status-code distribution, byte volume, and a top-domains ranking. Per-node only; cluster aggregation deferred.
Collection design	SHIPPED	nginx writes a JSON-escaped stats log; a background loop tails it incrementally, aggregates by (<code>domain</code> , <code>minute</code> , <code>status</code>) , and persists its read offset so a restart cannot double-count. Raw per-request rows are never stored — the aggregate is bounded, the raw log is not, and <code>runtime/</code> is disposable anyway. Collection sits outside the request path (no <code>log_by_lua</code>), so it cannot add latency; the cost is up to one minute of reporting lag.
Retention	SHIPPED	7 days, pruned by the collector.
Cluster and node inspection	SHIPPED	<code>/v status</code> for local state; <code>/v nodes</code> for the peer table with RTT, config version, program version, and a marker on the effective bot node. Because heartbeats carry program version, <code>/v nodes</code> is also the authority on which nodes have actually adopted a new build.
Certificate alerts	SHIPPED	Pushed to the group with the raw ACME error text, deduplicated over 24 h. The raw error is included on purpose: a sanitised message is a message that cannot be pasted into a search engine.

5.6 Operator interface

Capability	Status	Detail
Telegram control	SHIPPED	Group-only; direct messages refused. Five-category inline-keyboard menu (domains, rules, nodes, routing & network, AI) whose buttons emit real command text — so the menu teaches the language instead of hiding it.
Content-sniffing uploads	SHIPPED	Dropping a file into the group routes it by content, not by caption: PEM certificate, PEM key, or a configuration file. Config imports are parsed and summarised for review before anything executes.
Command / conversation split	SHIPPED	<code>/</code> -prefixed messages execute; <code>@</code> -mentions and replies to the bot go to the AI assistant. The split is strict in one direction that matters: a reply to the bot is <i>never</i> executed as a command regardless of its content, so conversational text cannot be mistaken for an instruction.
AI assistant	SHIPPED	Six interchangeable providers behind one interface — OpenAI, Anthropic, Gemini, DeepSeek, Grok, Qwen — configured at runtime with <code>/w ai</code> , with per-user conversation state. Strictly read-only analysis plus command <i>suggestion</i> : proposals arrive in fenced blocks rendered as explicit execute/cancel buttons. The model never has an execution path; a human press is always the last step.
CLI parity	SHIPPED	<code>cdn-agent exec "<command>"</code> runs the identical pipeline with non-zero exit codes and <code>--force</code> in place of <code>/confirm</code> . The cluster remains fully operable over SSH with Telegram unreachable.
One-line install	SHIPPED	Detects the distribution, installs OpenResty — racing the upstream repository against a domestic mirror and taking whichever responds first, for nodes with poor international connectivity — provisions both systemd units, generates the 100-year self-signed fallback certificate, and optionally joins an existing peer.

6 Case Study: Making Upgrades Honest

This is the clearest illustration of principle 7 — the program's boundary is the information boundary — and of how an operator complaint was traced to an architectural flaw rather than patched at the symptom.

The report

A one-command cluster upgrade reported success, yet several nodes remained on the old build. The operator discovered this only by logging into each machine individually and fixing them by hand — precisely the manual work the feature existed to eliminate.

The root cause

The `/mesh/upgrade` endpoint acknowledged the request and *then* began downloading, replacing, and restarting asynchronously. Every failure after the acknowledgement — a failed download, a checksum mismatch, a service that would not come back — was invisible to the initiating node. The command had never reported that the upgrade succeeded. It had reported that the request was accepted, while displaying that as success. The bug was not in the download logic; it was in what the system claimed to know.

The fix: observable

The initiator now polls peer heartbeats for the new `program_version` for up to 90 seconds and returns a real report: *N of M nodes confirmed, with a named failure list*. Confirmation is derived from the same heartbeat field that `/v nodes` displays, so there is exactly one source of truth about which node runs which build, and an operator can re-check it at any time without re-running anything.

The fix: reversible

Before replacing its binary, each peer backs up the previous one. The replacement is supervised by a watchdog launched as an *independent* systemd transient unit — necessarily independent, because a watchdog inside the agent's own cgroup would be killed by the very restart it is supposed to observe. The watchdog restarts the agent and requires three consecutive `is-active` checks within 60 seconds; a crash loop fails that window and triggers an automatic restore of the previous binary.

THE HONEST CAVEAT, DOCUMENTED IN THE DESIGN

The rollback watchdog lives in the binary *currently running* on the receiving peer. Upgrading from a version that predates this work therefore gets verification and reporting, but no rollback — protection begins with the following upgrade. This limitation is written into the design document rather than left for a future operator to discover during an incident. Stating a guarantee's start date is part of the guarantee.

7 Consistency Model and Security Posture

7.1 Consistency

MeshCDN is an eventually consistent system, stated plainly rather than implied. The guarantees it does make:

- A batch is atomic. Writes and the version increment share one SQLite transaction.
- Failed commands never enter the protocol. Any version a peer can pull describes only confirmed-successful state.
- Version numbers increase monotonically; a peer only ever pulls forward.
- Convergence is typically sub-second via push and bounded by roughly two heartbeat rounds otherwise.
- Configuration sync is durable and ordered; event-stream messages (alerts, challenges) are best-effort by design and backstopped by higher-level retries.

What it does *not* guarantee: linearisability, or conflict resolution for simultaneous contradictory writes on two nodes. For a system where writes originate from one operator through one chat group, that trade buys the removal of consensus machinery entirely.

7.2 Security

- **No management surface.** No web panel, no public API, no dedicated admin port. Mesh endpoints share the CDN's HTTPS listener and are indistinguishable from ordinary traffic to an outside observer.
- **Authorisation by shared secret.** Cluster membership requires `sha256(group_id + bot_token)`; the group binding means chat-level access control doubles as cluster access control.
- **Restricted control channel.** Group-only command acceptance; direct messages refused; the chat log is the audit log.
- **Blast-radius limits.** Confirmation gates on destructive and cluster-wide-conflicting operations, with short-lived non-persistent tokens.
- **Input hardening.** Closed parameter key sets, regex validation against ReDoS shapes, Lua string escaping on every value interpolated into generated configuration, and challenge-token format validation on the event stream.
- **Least contact.** Non-bot nodes never connect to any third-party API — a privacy property and a censorship-resistance property at once.

8 Engineering Practices

8.1 Source self-containment

A released binary is required to be able to reproduce its own source. The build embeds a snapshot of the tree via `go:embed` and stamps in the commit hash and build time; `cdn-agent dump-source` writes the complete tree back out, and release archives ship a flat source copy as a second guarantee. The build discipline is absolute: *a binary without embedded source is not releasable*. For infrastructure that may run untouched on a remote machine for years, "which code is this box actually running" must be answerable from the box itself.

8.2 Dependency minimalism

Three direct dependencies: an ACME client, a Telegram client, and a SQLite driver. Nginx configuration generation, the precedence engine, the mesh protocol, the certificate store, the statistics pipeline, and the command system are all first-party. Each dependency is at a genuine external boundary — a protocol or a storage engine — rather than an abstraction that could have been written in a few hundred lines.

8.3 Structured error taxonomy

Every command failure carries a stable short code — `BAD_FORMAT` , `UNKNOWN_TYPE` , `BAD_PARAMS` , `NOT_FOUND` , `CONFIRM_REQUIRED` , `CONFIRM_EXPIRED` , `CONFIRM_UNKNOWN` , `PORT_CONFLICT` , `CASCADE_BLOCKED` , `INTERNAL` . Front-ends render codes as they see fit: friendly prose and emoji in chat, exit codes and stderr in the CLI. The classification is decided once, in the core, and never re-derived by string matching at the edges.

8.4 Design document as constitution

The project maintains a versioned design document that is explicitly the final authority for implementation decisions, including a change history, a table of deliberately deleted concepts, and a "do not start without sign-off" list for features whose design is unresolved. Deferred work carries its reason: an unpicked vendor API, an unbudgeted operational cost, a missing fallback strategy. Each shipped task records what was verified and what remains to verify in a real multi-node environment — the difference between "tested on a laptop" and "tested in production" is written down rather than assumed.

8.5 Change discipline

Work is tracked in a task book with explicit acceptance criteria, and each entry records root cause, files touched, verification performed, and residual risk. Several entries in the log are self-initiated corrections found during review — the alphabetical size-merge bug, a shutdown race that could drop one statistics interval, a regex-engine mismatch that was analysed and then *consciously kept* with a documented rationale rather than changed reflexively.

9 Deployment Profile

Runtime requirements		Operational parameters	
Operating system	Linux with systemd; Debian/Ubuntu and RHEL families auto-detected	Heartbeat interval	60 s
Architecture	amd64	Sync latency	< 1 s typical (push); < 2 min worst case (heartbeat)
Data path	OpenResty (installed by the installer if absent)	Certificate scan	every 6 h
Control path	cdn-agent , one static binary, ~5 MB	Renewal threshold	3 days remaining
Services	meshcdn-nginx.service , meshcdn-agent.service	Alert deduplication	24 h per identifier
State	/etc/meshcdn/{persistent, runtime}	Upgrade confirmation	up to 90 s of heartbeat polling
Inbound ports	CDN traffic ports only; mesh is multiplexed onto them	Rollback health window	3 × is-active within 60 s
External egress	ACME directory; Telegram API from the bot node only	Stats granularity / retention	1 minute / 7 days
		Confirmation token TTL	5 min, in memory
		Self-signed validity	100 years, generated at install

9.1 Bringing up a cluster

```
# Node 1 – becomes the bot node by join order
curl -fsSL <release>/install.sh | bash -s -- \
    --bot-token <token> --group-id <group>

# Nodes 2..N – join by presenting the shared secret to any live peer
curl -fsSL <release>/install.sh | bash -s -- \
    --bot-token <token> --group-id <group> --peer <ip-of-any-node>

# From the Telegram group – configure the whole cluster in one atomic batch
/w domain https://cdn.example.com:443 https://10.0.0.4:443
/w cache static-7d patterns=*.jpg,*.png,*.css,*.js ttl=604800 browser_ttl=86400
/w defense no-bots ua=(curl|wget|python-requests) action=block
/w bind cdn.example.com cache:static-7d
/w bind cdn.example.com defense:no-bots

/v nodes - - # confirm every node reached the new config version
```

10 Release History and Direction

Version	Substance
v1.x	Single-node reverse proxy with Telegram control; the command-language concept established.
v2.x	Mesh synchronisation, certificate subsystem, AI assistant integration.
v3.1	Rule templates and domain groups. Their friction motivated the v4 rewrite.
v4.0	Full rewrite on the strict four-segment grammar, object/binding model, single version stream, snapshot pull, dual-directory persistence, and source self-containment.
v4.0.20	Working UA filtering with ReDoS rejection and closed key sets; multi-SAN renewal; cluster cache purge; bot-role transfer at the state layer; per-node traffic statistics.
v4.3.0	Current. Verified, rollback-capable cluster upgrades; strictness-preserving size merges; graceful shutdown of the statistics collector.

10.1 Next, and why not yet

- **Smart origin routing** — needs a signed-off per-hop fallback strategy first; without it, relaying trades one failure mode for a worse one.
- **Cross-node statistics aggregation** — the per-node pipeline is built and the roll-up is a straightforward extension; deferred until demand justifies the query cost.
- **Restart-free bot handover** — requires a supervisor refactor to start and stop Telegram polling in place.
- **GeoIP and CC protection** — both impose recurring operational costs (database updates, shared-memory state) the operator must accept explicitly.
- **DNS-01 issuance and DNS health eviction** — both require committing to specific provider APIs.

The consistent pattern: features are deferred for unresolved *design* questions, not unresolved implementation. Shipping a relay without a fallback path, or a GeoIP database nobody has agreed to update, would satisfy a checklist and degrade the system.